



UNIVERSITY OF GENEVA

Faculty of Science

Assignment 3

Computational Imaging
14x062



Michel Jean Joseph Donnet

2026-04-06



Contents

1	Introduction	3
2	Task 1: Depth estimation	3
2.1	Features	4
2.1.1	Color features	4
2.1.2	Texture features	4
2.1.3	Edge features	4
2.2	Results	7
3	Task 2: Image refocusing	8
3.1	Refocusing using depth map	8
3.2	Refocusing using sub-aperture images	9
4	Task 3: Image refocusing and depth estimation	11
5	Task 4 (optional): Refocusing and depth estimation with neural networks	14
6	Conclusion	15

1 Introduction

The classical digital photography allows to capture images with a good quality. However, due to the physical properties of the camera, the focus is made on a specific point, and the notion of depth of field appears: objects not focused are blurred.

To respond to this problem, computational imaging allows to refocus the image after it has been captured thanks to analysis of the depth of field. Indeed, the depth of field gives a notion of the distance between the camera and the objects of the scene, and can be estimated by an analysis of the blur of the image, as we will study in the first part of this work. Then, thanks to the depth of field information, a refocus can be performed thanks to application of specific filters according to the depth of field, as we will see in the second part of this work.

To have a more precise estimation of the depth of field, some cameras have been designed like the plenoptic camera. These cameras capture the depth of field by a multi view acquisition system created by a microlens array placed in front of the sensor. The depth of field can then be estimated by an analysis of the multi view acquisition and the refocus can be performed in this way, as we will see in the third part of this work.

2 Task 1: Depth estimation

The code was implemented in matlab. As some random forest was used in matlab, the code wasn't directly reusable in python.

That's why I try to understand the code with the main functions and principles. Then, I used the understanding to implement from scratch a similar pipeline in python using ChatGPT and python libraries like scikit-learn.

The base principle of the depth map estimation is described in the Figure 1 for the training pipeline, and in Figure 2 for the prediction pipeline.

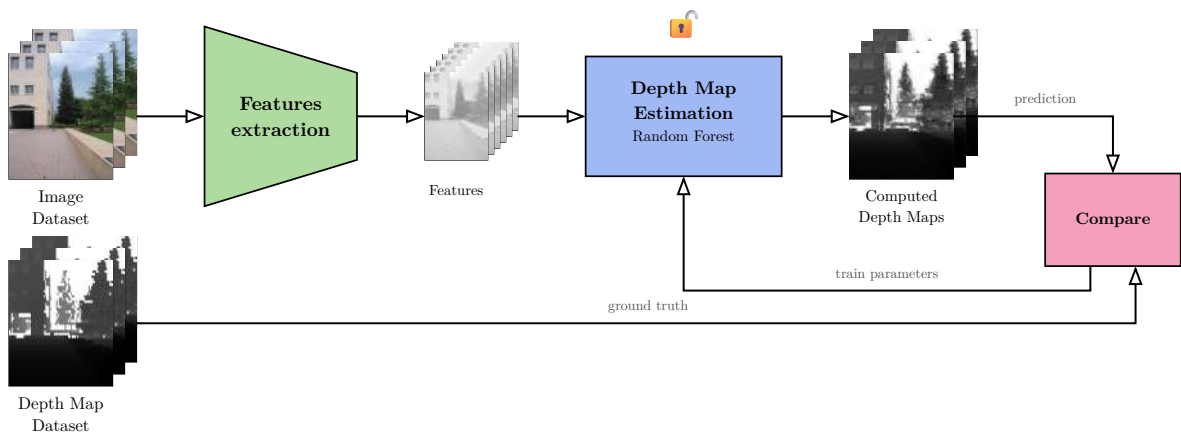


Figure 1: Depth map estimation training based on the given matlab code

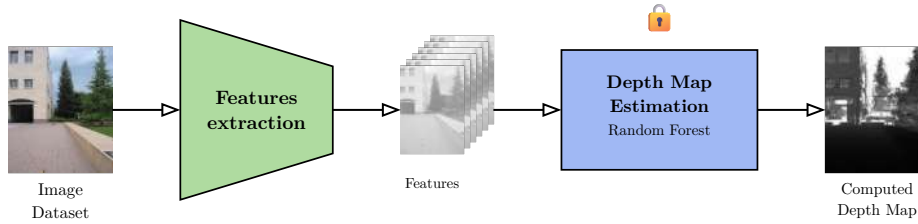


Figure 2: Depth map estimation prediction

2.1 Features

There is a lot of features extracted used to compute the depth map, such as color, edge and texture features.

2.1.1 Color features

As shown on Figure 3, color features are extracted from different color spaces:

- RGB (Red, Green, Blue)
- LUV (Lightness, U and V chrominance channels)
- HSI (Hue, Saturation, Intensity)

They allow to capture different representations of the image that are used in depth estimation. Indeed, an object with a specific color is more likely to be at a specific depth.

2.1.2 Texture features

Texture features are computed by applying laws filters to the image, as shown on Figure 4.

Laws filters are a set of filters designed to capture different levels of texture in an image.

Indeed, some of the filters make a smoothing of the image while others capture fine details.

2.1.3 Edge features

Edge features are computed by applying Navatia-Babu filters to the image, as shown on Figure 5.

Navatia-Babu filters are a set of filters designed to capture edges in an image at different orientations and scales. The current implementation uses 6 filters to capture edges at orientations of 0° , 30° , 60° , 90° , 120° and 150° .

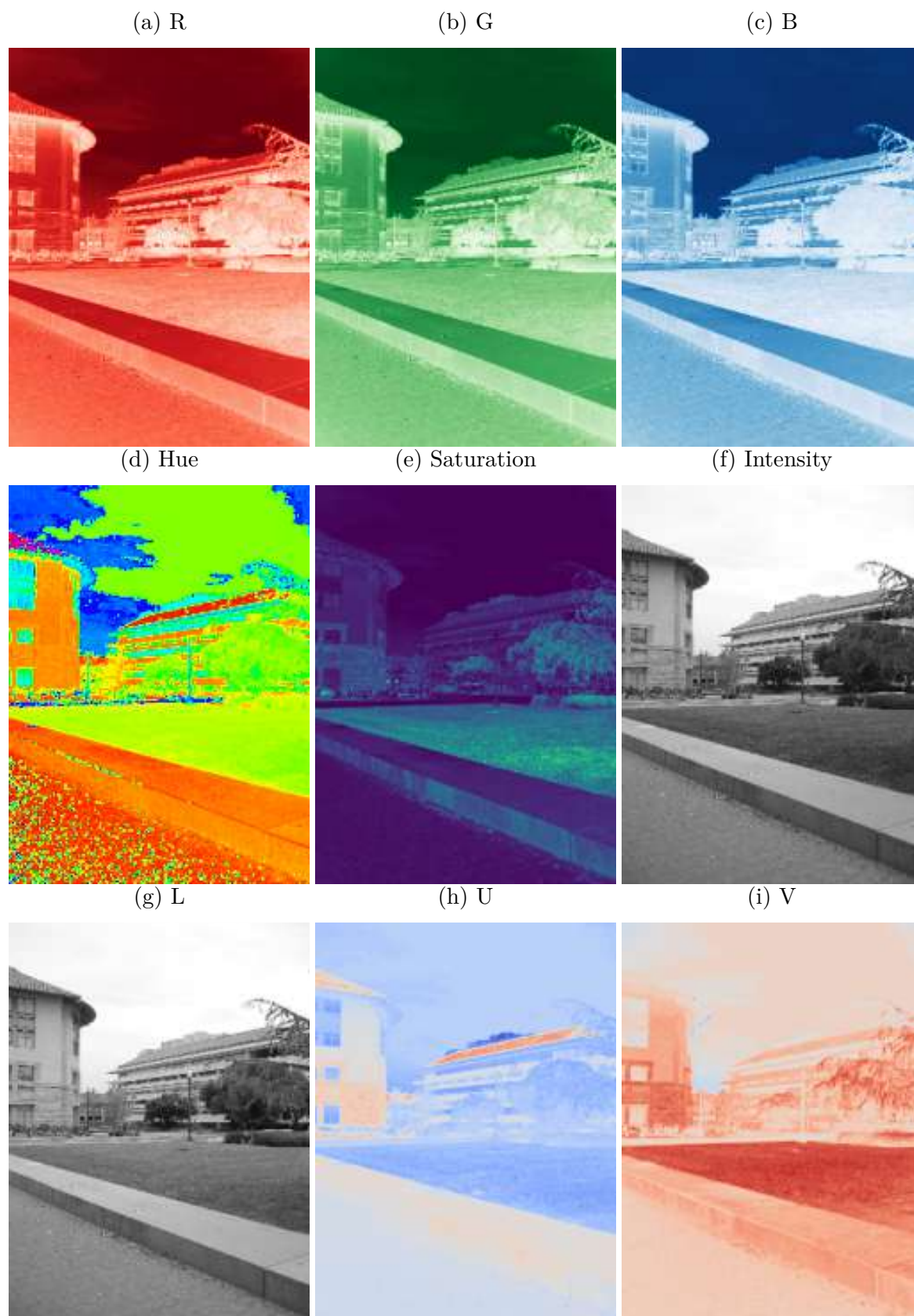


Figure 3: Color features

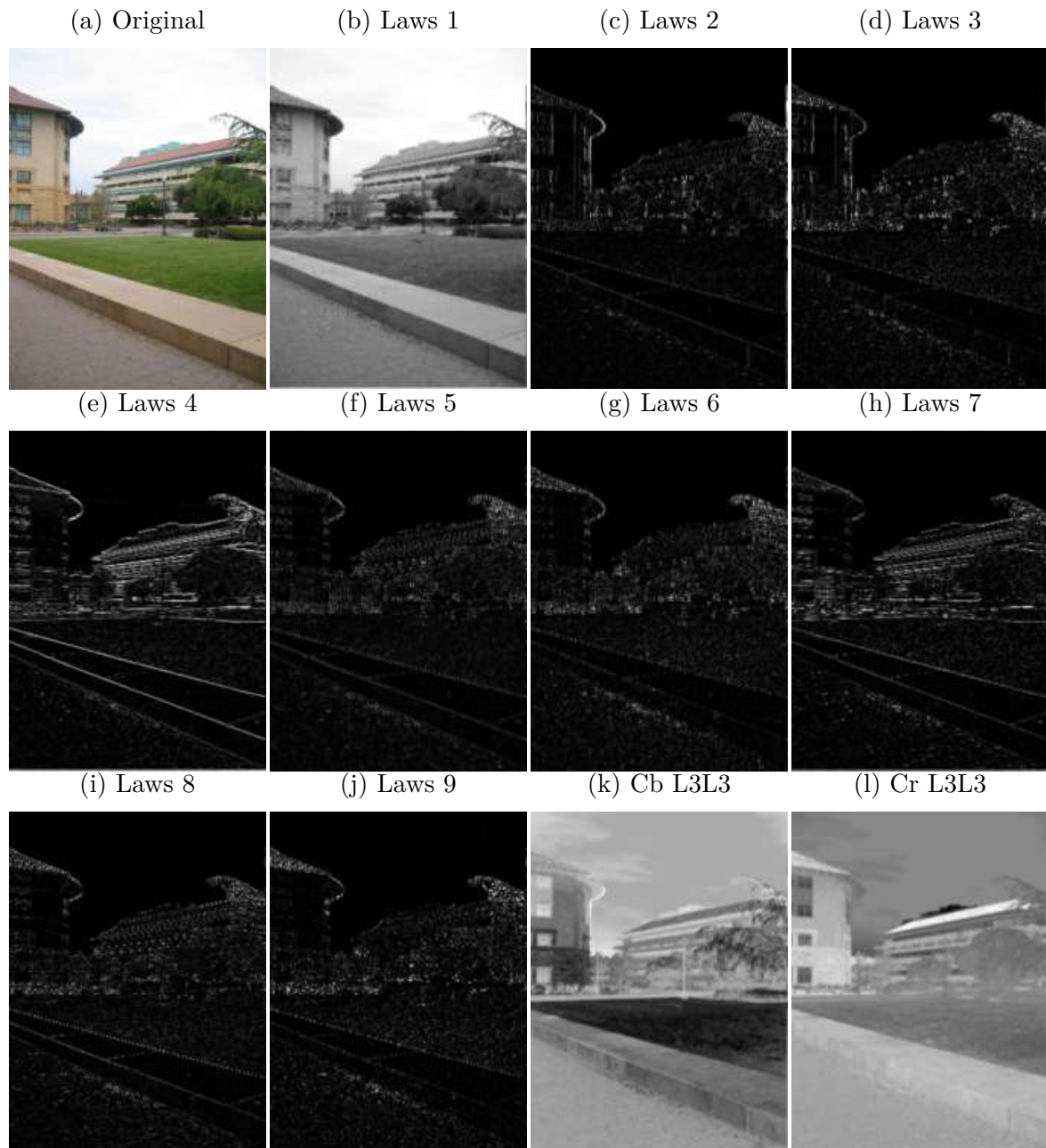


Figure 4: Laws texture features

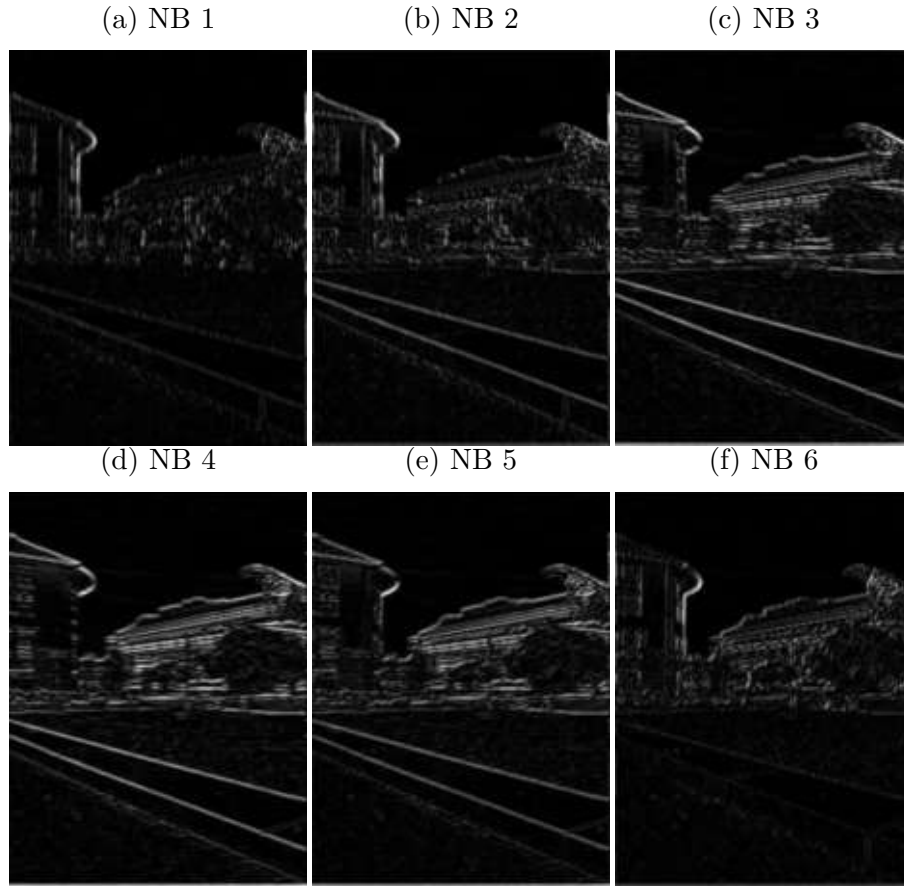


Figure 5: Navatia-Babu edge features

2.2 Results

The results obtained with the implemented pipeline are shown on Figure 6.

We can see that the predicted depth map captures the general depth of the scene, even if it is not perfect and noisy. Indeed, the depth of the sky is correctly estimated to be far while the depth of the ground is correctly estimated to be close.

However, as we can see, the depth is quite noisy and not very accurate, which is due to the single capture of the image and the complexity of the depth estimation problem that requires a lot of information to be accurate, like for instance a focal stack or a focus-aperture stack or a multi view acquisition as in plenoptic cameras. On top of that, the database used for training has very small and pixelized depth maps, which makes the training not very good and the results not very accurate. And the computational power of the model is also quite limited, which makes the results not very good.

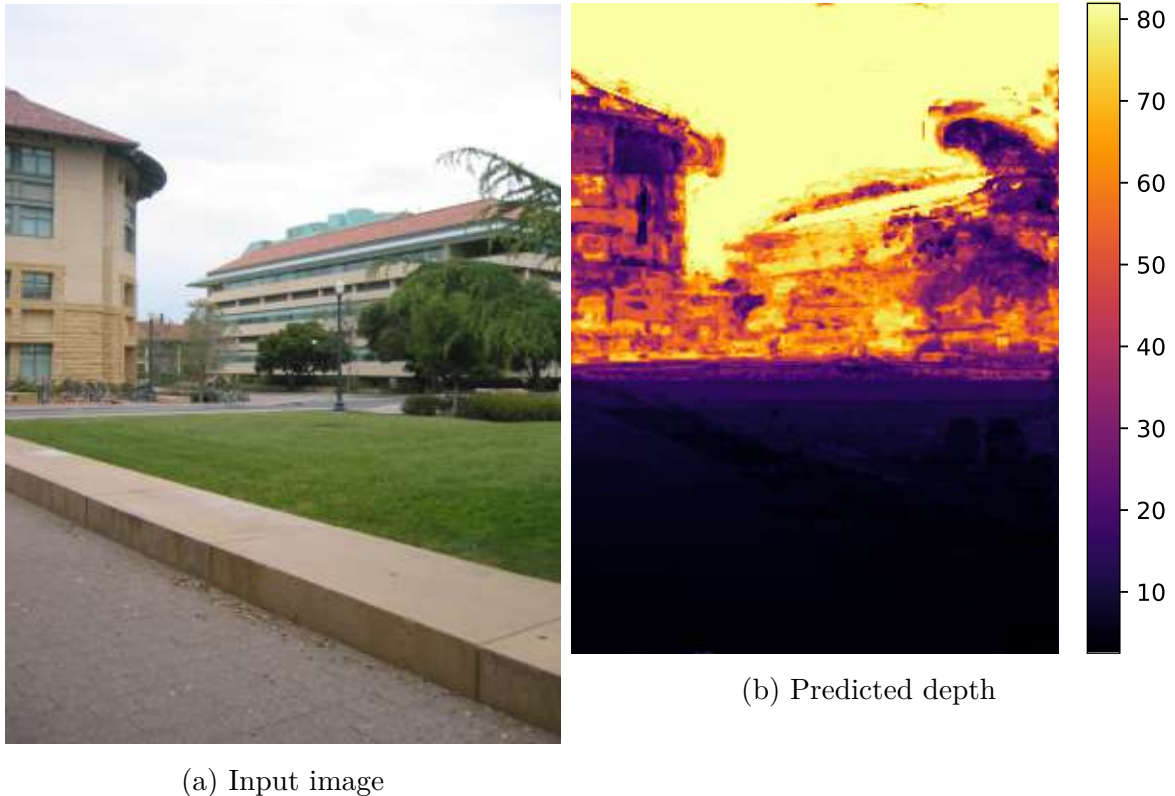


Figure 6: Depth estimation results

3 Task 2: Image refocusing

In this task, we will use the depth map of an image or some sub-aperture images to perform some refocusing of the image, based on an existing implementation in matlab. To convert the matlab code to python, I used another approach than in task 1: this time, I converted the matlab code to python line by line.

Then, I used Github Copilot to update the documentation to numpy style and to debug the code to make it properly working in python.

3.1 Refocusing using depth map

For the refocusing using depth map, the principle is to select the pixels of the image that are at a specific depth and apply a blur filter that increases with the distance to the selected depth on other pixels, as shown on Figure 7.

The obtained result is a refocused image at the selected depth, as shown on Figure 8. Thanks to the depth map, the refocusing is accurate and the result is quite good. Indeed, the information of the depth is crucial to perform a good refocusing, as it allows to apply the right amount of blur on each pixel according to its depth to have a realistic refocus that follows the physical properties of the camera.

So thanks to the depth map, a good refocusing can be performed after the capture of the image, which can be only obtained thanks to computational imaging techniques.

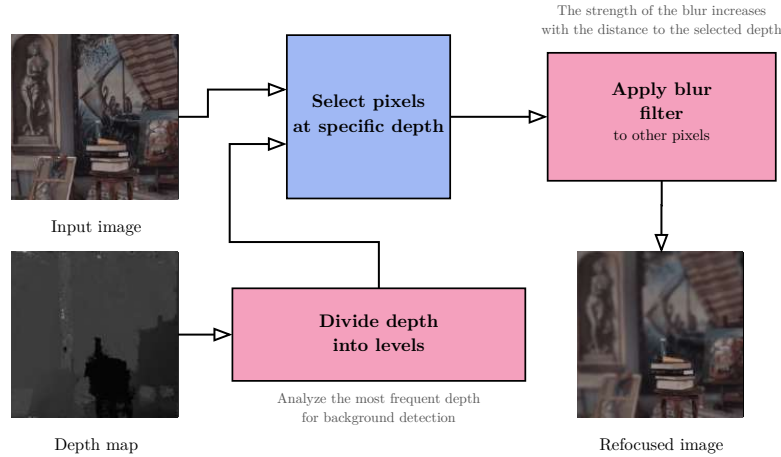


Figure 7: Refocusing using depth map

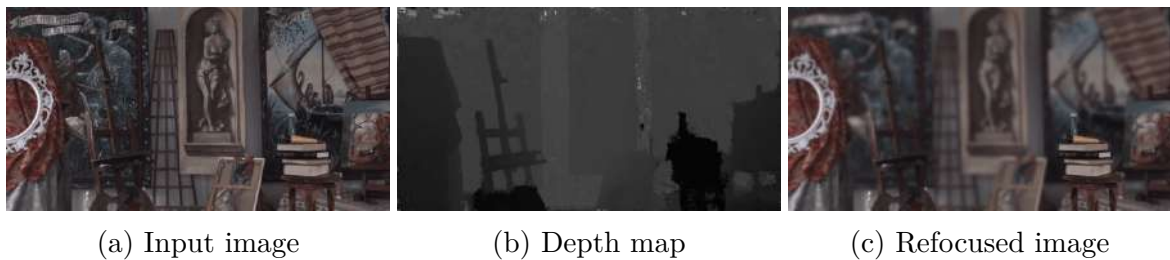


Figure 8: Refocusing on the books using depth map

3.2 Refocusing using sub-aperture images

The principle of the refocusing using sub-aperture images is to use the multi view acquisition. The multi view acquisition can be obtained by multiple cameras that takes the same scene from different angles. It can also be obtained by a camera on a moving platform that takes multiple captures of the same scene from different angles. Or the multi view acquisition can be obtained by a camera that has a microlens array placed in front of the sensor allowing to capture the scene from different angles in a single capture, as in plenoptic cameras.

Then, the shift and sum refocusing technique can be obtained by a combination of all views of the multi view acquisition. Some views are shown on Figure 10. By showing the circle on the left of the images, we can see that the circle is at different positions in each view, which confirms that the views are taken from different angles. The views must then be aligned according to the desired refocus depth, and then summed together to obtain the refocused image. So there is no need of a depth map to perform the refocusing.

If we look at the results in Figure 11, we can see that the refocusing gives a good result at the refocused depth. However, the blur is not very realistic due to the multi view acquisition: indeed, the blur is like a squared blur since all acquisitions are aligned and summed together with different angles, making the blur not very natural. This is probably due to the fact that the multi view acquisition is made from different positions, which makes the alignment of the views not perfect and therefore the blur not very realistic.

So this refocusing technique works quite well even if the result is not very realistic: in fact, this method can be improved to have a more realistic blur. For instance, thanks to the multi view acquisition, a depth map can be easily estimated, allowing to perform therefore

a better refocusing by applying different blur filters according to the depth map, as seen in the previous part of this work.

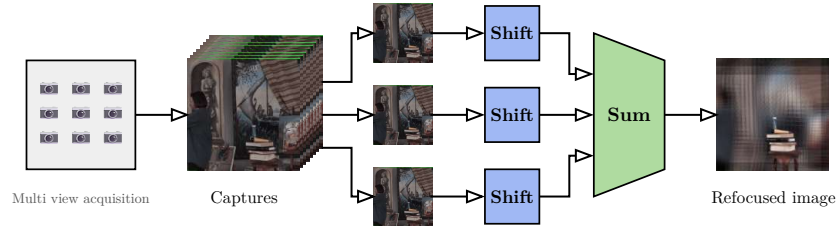


Figure 9: Refocusing using sub-aperture images

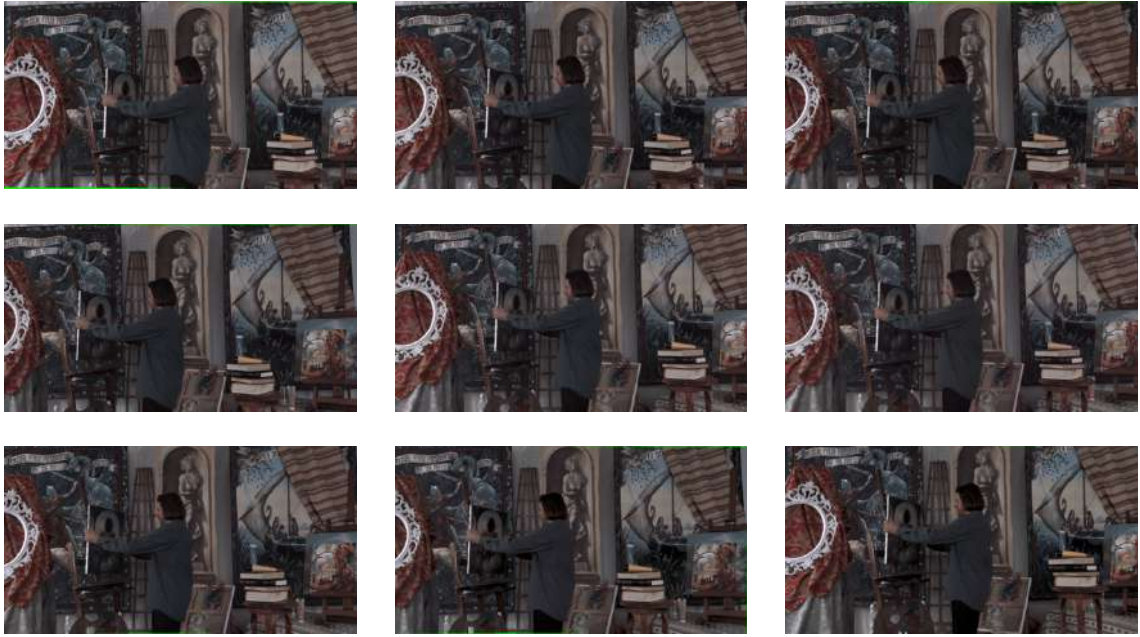


Figure 10: Sub-aperture images

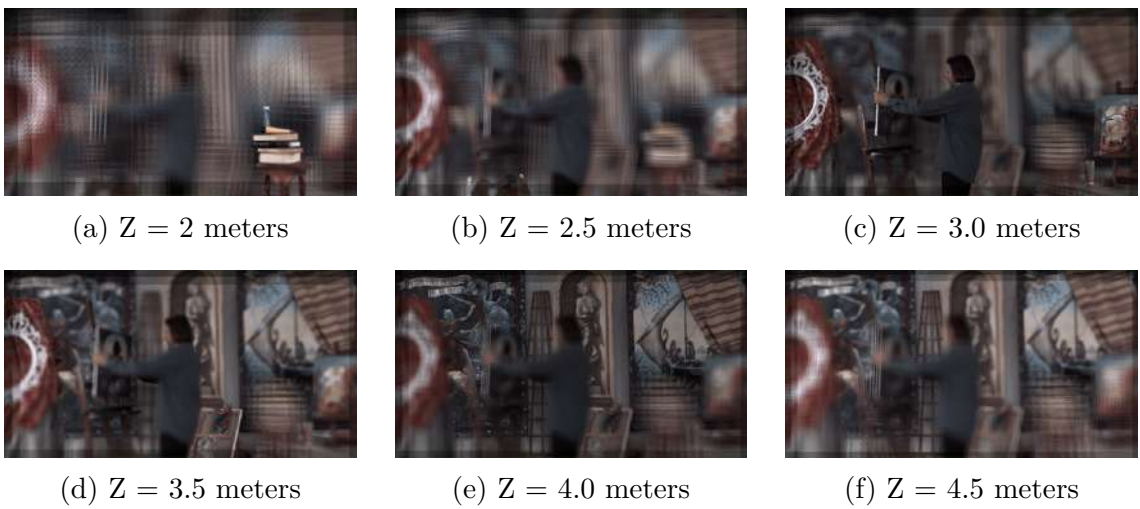


Figure 11: Refocused at Different Depths



4 Task 3: Image refocusing and depth estimation

In this task, we will study some multi view acquisition from a plenoptic camera taken from the github repository <https://github.com/hahnec/plenoptcam>.

As the code was already delivered in the repository as a jupyter notebook, I only played with the code and the parameters to understand how it works and how the refocusing and depth map estimation can be performed with the multi view acquisition of a plenoptic camera. However, by trying by myself to run the code, I had some issues with libraries and dependencies. Indeed, the code was implemented with older libraries and some functions were deprecated, making the code not directly runnable.

The main principle of the plenoptic camera is to capture the scene from different angles thanks to a microlens array placed between the lens and the sensor, allowing to capture multiple views of the scene in a single capture. The result is a multi view acquisition that can be exprimed as a 4D matrix (x, y, u, v) where (x, y) are the spatial coordinates and (u, v) are the angular coordinates of the light field.

The obtained multi view acquisition must then be processed to have a good representation of the scene. Indeed, without processing, the output looks like an image with a lot of small squares, as shown on Figure 12a.

To process the image, the disposition and properties of the microlens array must be known, so a calibration step is necessary to perform good processing of the image. Indeed, if some zoom was applied, the centroid of the microlenses is not at the same position than estimated, so the results can look messy. To calibrate, some white images of calibration are available, as shown on Figure 12b. Then, the calibration image taken must have same parameters than the image to process.



(a) Raw Illum image

(b) Raw white calibration image

Figure 12: Raw Illum image

When the calibration is done, each sub-aperture image can be extracted from the multi view acquisition, as shown on Figure 13.



Figure 13: All sub-aperture images view

Then, thanks to the multi view acquisition, we can take the view of the scene from a specific angle as shown on Figure 14. In fact, it corresponds to take all pixels of the multi view acquisition that corresponds to a specific angle, which is like taking all pixels of a pinhole camera with a very small aperture that captures only the light coming from that specific angle, allowing to have a large depth of field. However, the price to pay is that the image is with a low resolution since only a portion of the pixels is used to create the image. For instance, in our case, as we have around 100 views, the resolution of each view is around $1/100$ of the overall resolution of the sensor.

As we can see on Figure 14, the difference between views is very small (we can see that at the borders of the images). Indeed, the acquisition is made with a microlens array, which means that the views are taken from close angles, making the difference between views quite small. That's why to have good results, many views must be taken.



(a) Left view

(b) Central view

(c) Right view

Figure 14: Left view

Thanks to this multi view acquisition with a large depth of field, a depth map can be estimated by an analysis of the disparity between views, as shown on Figure 15a.

Indeed, an object that is close to the camera will move more between views compared to an object far from the camera, allowing to estimate distances of objects in the scene and therefore to estimate a depth map of the scene.

Thanks to these properties, a refocus can also be performed by a shift and sum of the views, as shown on Figure 15b. Compared to the shift and sum refocus performed in task 2, the refocus performed with a plenoptic camera is more accurate with a better blur. This is probably due to the fact that the multi view acquisition is made from a single capture at a single position with a microlens array, which allows to have a better alignment of the views and therefore a better refocus. It is also possible to perform a refocus like in task 2 by applying different blur filters according to the depth map estimated, or to perform a refocus using different method such as Scheimpflug refocusing that allows a refocus of an inclined plane in the scene, such as the ground for instance, which is not possible with the shift and sum refocusing technique, or with classic camera focusing. The Scheimpflug refocusing corresponds to a shift and sum refocus with an affine shift that depends on the position of the pixels.

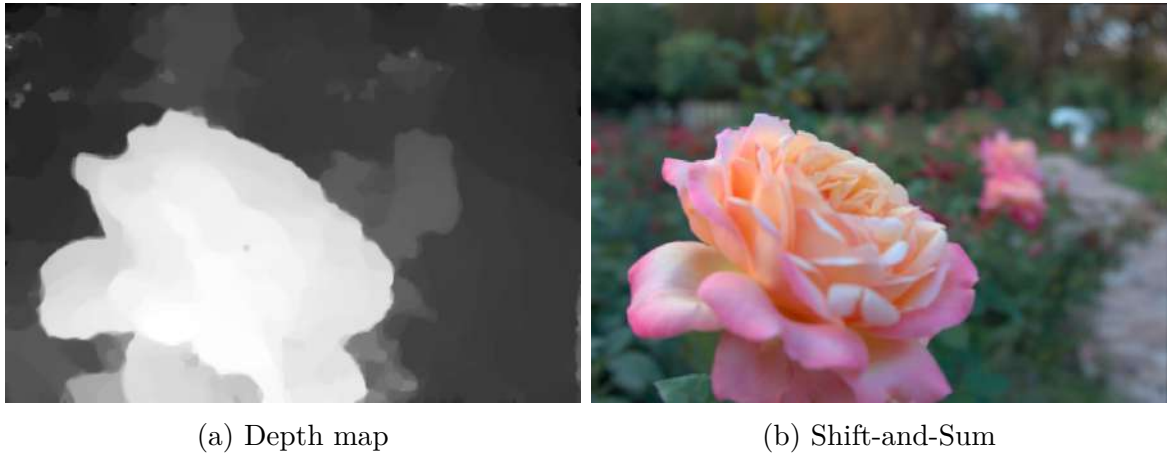


Figure 15: Depth map

So the pipeline of the processing of a multi view acquisition from a plenoptic camera can be summarized like in the Figure 16.

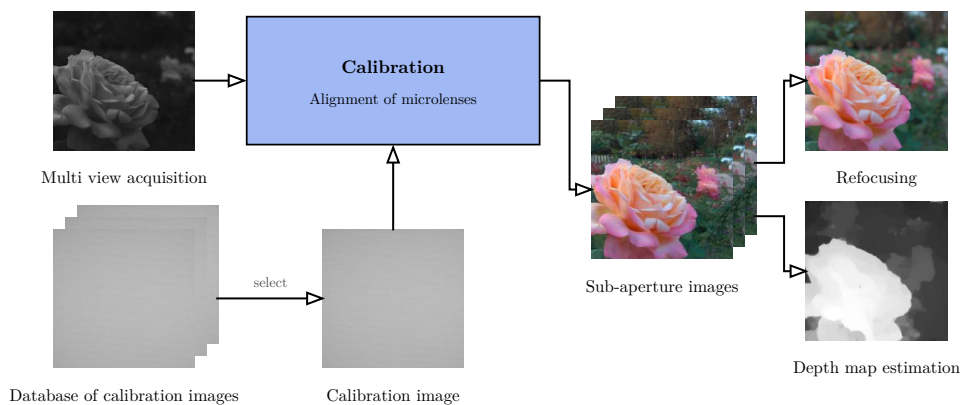


Figure 16: Pipeline of the processing of a multi view acquisition from a plenoptic camera

5 Task 4 (optional): Refocusing and depth estimation with neural networks

The depth map estimation can be made with a neural network instead of a random forest regressor. Indeed, the depth estimation is complex and requires a lot of information to be accurate, so a neural network is more suitable to capture relationships between features and depth. The general pipeline of the depth estimation with a neural network could be represented like in Figure 17. Note that the extraction of features is not necessary with a neural network: it can be handcrafted with fixed filters. This can allow to speed up the training or to have a smaller dataset, but it can limit performances if handcrafted features are not relevant enough.

So depending on the architecture of the neural network used, the training must be applied on the encoder and decoder together, or only on the decoder if the encoder is handcrafted with fixed filters.

The big advantage of using a neural network is that when the network is trained, the depth estimation can be very fast and accurate. However, the training of the network can be long and requires a lot of data and computational power, which can be problematic.

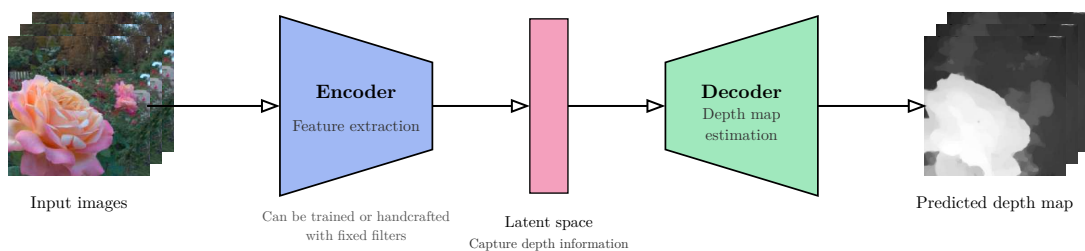


Figure 17: Pipeline of the depth estimation with a neural network

In case of refocusing, a neural network can also be built. The training can be made on single acquisition with a depth map, or on single acquisition without depth map, or on multi view acquisition, etc. In all cases, the principle stays the same: learning the relationship between the non-refocused input and the refocused image at a specific depth. Due to the wide variety of possible inputs and outputs, the architecture of the neural network can be very different from one case to another. For instance, if we dispose of a multi view acquisition, the relationship between the input and output will not be the same than if we only have a single acquisition with no depth map. Indeed, if we dispose already of a multi view acquisition, we have already a lot of features so only a decoder is necessary whereas with single acquisition with no depth map, an encoder is necessary to extract features. Note that it can also be handcrafted depending on the case: for instance, the features set can be some unfocused images computed with different filters or something else.

So the neural network can be very interesting for refocusing and depth estimation as they are able to learn complex relationships between inputs and outputs. However, the training can be long and requires a lot of data and computational power, which can be problematic.



6 Conclusion

In this work, we have implemented first a depth map estimation based on a single image by extracting colors, textures and edge features and processing them together with a random forest regressor to estimate the depth map of the scene. The results are not very accurate and quite noisy, which is due to the complexity of the depth estimation problem that requires a lot of informations and also due to the database wich contains only small depth maps for images and computational power.

Then, a study of refocusing techniques has been done, based first on depth map information and then on multi view acquisition. With the depth map, the refocusing is quite good and accurate, and the blur is realistic thanks to the depth information allowing to apply different blur filters according to the depth of the pixels. With the multi view acquisition, the refocusing don't need a depth map. However, due to the quality of the sub-aperture images, the blur is not very realistic even if the refocus is good at the refocused depth.

Finally, a study of the refocusing and depth estimation with a plenoptic camera has been done. The processing and calibration takes some time and the output image has smaller resolution than the sensor resolution, but the results are quite good and allows to change angle of view, refocus using different techniques, compute a depth map, etc. So the plenoptic camera is very interesting for computational imaging and offers a lot of possibilities in terms of post-processing.

The depth estimation and refocusing can also be performed using neural networks thanks to the ability of neural networks to learn complex relationships between inputs and outputs. But the training can be long and requires a lot of data that are not always available, and some computational power to obtain good results. That's why plenoptic cameras and simple algorithms stay very interesting in term of computational imaging, even if the results are not perfect.

We may wonder if the video allows good depth estimation and refocusing thanks to the multi frames capture, which provides much more information than a single capture.